

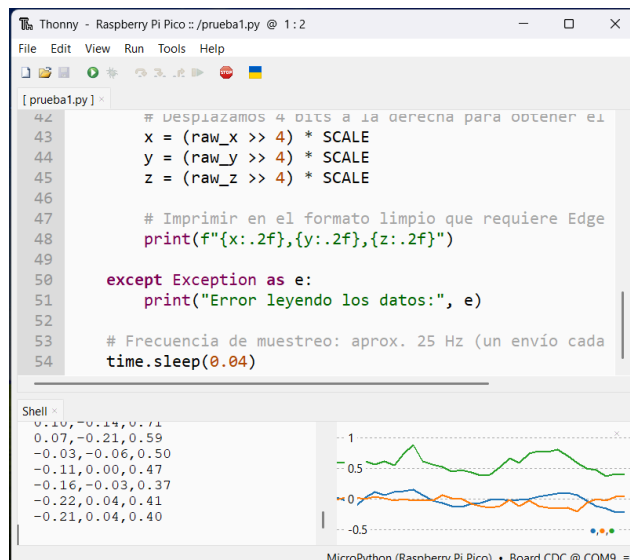
# Manual de Programador

## Introducción y Alcance

El propósito de este software es gestionar un sistema de alarma IoT dividido en dos capas que monitoriza el movimiento del usuario para activarse durante la fase de sueño ligero. El sistema busca mejorar la calidad del descanso evitando que la alarma suene en fases de sueño profundo. El alcance abarca la captura de datos brutos del acelerómetro, la transmisión serie hacia el ordenador, el cálculo de ventanas horarias y el disparo de un buzzer físico.

## Requisitos del Sistema y Dependencias

Para el correcto funcionamiento y desarrollo del proyecto se requiere: Entorno de ejecución de Python 3.x instalado en el ordenador de control. Librería externa pyserial, encargada de gestionar los flujos de lectura y escritura a través del puerto serie USB. Se puede instalar desde el gestor de paquetes pip con el comando `pip install pyserial`. Firmware de MicroPython cargado en la placa Raspberry Pi Pico para la interpretación de los scripts embebidos. Editor Thonny o similar para la carga del código en el microcontrolador.



```
[prueba1.py]
42     # Desplazamos 4 bits a la derecha para obtener el
43     x = (raw_x >> 4) * SCALE
44     y = (raw_y >> 4) * SCALE
45     z = (raw_z >> 4) * SCALE
46
47     # Imprimir en el formato limpio que requiere Edge
48     print(f"{x:.2f},{y:.2f},{z:.2f}")
49
50     except Exception as e:
51         print("Error leyendo los datos:", e)
52
53     # Frecuencia de muestreo: aprox. 25 Hz (un envío cada
54     time.sleep(0.04)
```

Shell

```
0.10,-0.14,0.71
0.07,-0.21,0.59
-0.03,-0.06,0.50
-0.11,0.00,0.47
-0.16,-0.03,0.37
-0.22,0.04,0.41
-0.21,0.04,0.40
```

MicroPython (Raspberry Pi Pico) - Board CDC @ COM9

## Arquitectura y Diseño

El sistema utiliza una arquitectura modular basada en comunicación por puerto serie síncrona y bidireccional. La lógica se divide en dos componentes independientes:

**Componente de Hardware (main.py):** Funciona como un nodo de captura y acción. Lee los registros de datos espaciales del acelerómetro mediante el protocolo I2C y los envía codificados en texto plano hacia el bus serie. Queda a la espera de un comando externo de activación y maneja la interrupción por hardware del botón integrado.

**Componente de Software (despertador.py):** Funciona como el motor lógico y de toma de decisiones. Escucha el puerto serie, parsea la información recibida y ejecuta el algoritmo de control en base al tiempo y al movimiento.

El algoritmo de detección de sueño ligero descarta el uso de inteligencia artificial local por problemas de dependencias, sustituyéndolo por un cálculo de magnitud geométrica escalar. El script procesa la cadena de texto de los tres ejes, los convierte a variables flotantes y

aplica la suma de sus valores absolutos. Si este valor escalar supera el umbral establecido de 1.25 dentro de la ventana de tiempo permitida, se genera el disparo.

### Instalación y Despliegue (Setup)

Para configurar el entorno de desarrollo desde cero, se deben seguir los siguientes pasos: Primero, conectar la Raspberry Pi Pico al ordenador mediante el cable USB. Segundo, abrir el entorno Thonny, abrir el archivo main.py y guardarlo directamente dentro del directorio raíz de la placa Raspberry Pi Pico. Tercero, colocar el archivo de script despertador.py en una carpeta local de tu ordenador. Cuarto, abrir el archivo despertador.py con cualquier editor de texto y modificar la constante global PUERTO\_COM asignándole el nombre del puerto que el sistema operativo le haya dado a la placa, por ejemplo COM9 en sistemas Windows o /dev/ttyACM0 en entornos Linux. Quinto, ejecutar el script del ordenador desde la consola del sistema operativo ejecutando el comando python despertador.py.

### Estructura de Directorios

La estructura del repositorio es limpia y consta de los siguientes archivos en la raíz del proyecto: main.py: Archivo que contiene el firmware en MicroPython que debe ser transferido a la memoria interna de la Raspberry Pi Pico. despertador.py: Script principal en Python 3 que debe ejecutarse en el sistema operativo del ordenador. README.md: Archivo de documentación general y presentación del proyecto para la plataforma de alojamiento.

### Variables de Configuración Clave

Para realizar tareas de mantenimiento o calibración del sistema, el programador debe modificar las siguientes variables declaradas como constantes al inicio del script despertador.py: HORA\_MINIMA y HORA\_MAXIMA: Definen las variables de tipo cadena de texto que delimitan la ventana de tiempo en la que se permite que el despertador se active si detecta movimiento de sueño ligero. UMBRAL\_MOVIMIENTO: Valor numérico flotante que determina la sensibilidad del sistema. Un valor más alto requerirá movimientos más bruscos del usuario en la cama para activar el buzzer, mientras que un valor más cercano a 1.0 aumentará la sensibilidad ante giros leves.

```
import serial
import time
import datetime

PUERTO_COM = 'COM9'
HORA_MINIMA = "07:30"
HORA_MAXIMA = "08:00"
UMBRAL_MOVIMIENTO = 1.25
MODULO_TEST_IGNORE_RELOJ = True

try:
    pico = serial.Serial(PUERTO_COM, 115200, timeout=1)
    pico.flushInput()
    pico.flushOutput()
    time.sleep(2)
except:
    exit()
```